

資料結構期末專題說明報告

題目:簡易密碼儲存伺服器

資電三 111504006 陳至恆

通訊二 112503018 陳立頌

目錄：

摘要	2
連線功能:	2
密碼強度檢測：	4
資料庫處理：	5
未來展望：	9

摘要：

由於網路上各種服務的帳號與密碼數量愈來愈多，我們打算自己設計一個能夠紀錄此資料的程式，不僅能深入了解密碼儲存的原理，也希望活用於作業中所獲得的開發程式庫相關的經驗，自行設計一個簡易的 json 資料庫，作為記錄各種網路服務的伺服器；並且結合基礎的連線功能，實現類似於網路查詢密碼，根據客戶輸入的指令，對欲改變的密碼進行讀取、修改或是刪除。

連線功能：

功能概述：

將記錄帳號密碼的檔案存放於伺服器一端，客戶端則是輸入介面提示的指令，像是取得目標服務的帳號與密碼；或是新增、刪除、修改密碼。便能根據使用者的不同而切換資料庫，而非侷限於單獨一位用戶。例如 AppleTV 通常變允許多個帳號共用優惠，而我們希望也能複製類似的功能，可能輸入指令的是相同的，但當讀取到目標服務時，僅顯示單獨的帳號、密碼(當然會先核對帳號擁有者的身分)，並非全部呈現(有違隱私問題)

技術細節：

運用課程「微算機原理」所學，實現簡易版 socket 功能，雖然是本地端連線，透過自定義 IP 以及 Port，使客戶端能夠順利連線伺服器的位址，並根據輸入的指令，取得想要的結果。其中伺服器必須先建立，並開始「監聽」，等待客戶的連線，因此在編譯上是會先編譯“server”檔案，等確定伺服器成功運作後，再將“client”檔案編譯，並於終端機輸入伺服器的地址（IP），若連線成功便會跳出相關的訊息。

```
508 int main(){
509     int server_fd, client_socket;
510     struct sockaddr_in address;
511     int addrlen = sizeof(address);
512     fd_set readfds;
513     struct timeval timeout;
514     char buffer[1024] = {0};
515     char *welcome_msg = "Connection established. Type 'exit' to disconnect.\n";
516
517     // 建立伺服器 socket
518     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
519         perror("Socket failed");
520         exit(EXIT_FAILURE);
521     }
522
523     // 設置 socket 選項
524     int opt = 1;
525     if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
526         perror("setsockopt");
527         exit(EXIT_FAILURE);
528     }
529
530     // 綁定地址和埠
531     address.sin_family = AF_INET;
532     address.sin_addr.s_addr = INADDR_ANY;
533     address.sin_port = htons(PORT);
```

圖一：位址、埠的設定

```

540 // 開始監聽
541 if (listen(server_fd, 3) < 0) {
542     perror("Listen");
543     exit(EXIT_FAILURE);
544 }
545 printf("Server is listening on port %d...\n", PORT);
546
547 // 接受客戶端連線
548 if ((client_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)) < 0) {
549     perror("Accept");
550     exit(EXIT_FAILURE);
551 }
552 printf("Client connected.\n");
553
554 // 傳送歡迎訊息
555 // send(client_socket, welcome_msg, strlen(welcome_msg), 0);
556
557
558 // 保持連線直到接收到 'exit' 或超時

```

圖二：確認連線過程順利

```

16 int sock = 0;
17 struct sockaddr_in serv_addr;
18 char buffer[1024] = {0};
19
20 // 建立客戶端 socket
21 if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
22     perror("Socket creation error");
23     return -1;
24 }
25
26 // 設置伺服器地址
27 serv_addr.sin_family = AF_INET;
28 serv_addr.sin_port = htons(PORT);
29
30 // 轉換伺服器 IP 地址
31 if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
32     perror("Invalid address/Address not supported");
33     return -1;
34 }
35
36 // 嘗試連線
37 if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
38     perror("Connection failed");

```

圖三、客戶端 Socket 的 IP 位址、埠設置

測試結果：

```

Enter commend: exit
Disconnecting from server...

Client disconnected.
Server shut down.

```

圖四、取消連線測試

修改過程、試錯紀錄：

Socket 的語法選項設計 `setsockopt()` 的選項組合問題，將 `SO_REUSEADDR` 跟 `SO_REUSEPORT` 同時設置時，會導致部分系統，如蘋果電腦的作業系統無法正確處理，出現“Protocol not Available”的錯誤訊息，這個問題讓我們理解到不

同作業系統在處理 socket 連線時，有各自的邏輯或是細節，同時使我們更加留意作業系統的差異，以免花太多時間在處理這種系統端的阻礙。

密碼強度檢測：

功能概述：

根據隨機性、字串（密碼）長度、字符的種類（大、小寫、數字、特殊符號共計四種）以計算密碼的強度，因此後來才改成上述的條件，其中長度必須大於 8 字元，否則一律視為弱的密碼！雖然後來沒有引用霍夫曼樹或是霍夫曼編碼的平均長度作為密碼強度的指標，但在嘗試的過程中更加熟悉了無失真資料壓縮的邏輯，透過自定義結構、搭配遞迴的觀念，我們確認了最後的編碼確實能依據各字元出現的頻率高低進行合理的編碼分配，但由於密碼本身的長度偏低，因此壓縮過的編碼效益並沒有顯現出來。

修改過程、試錯紀錄：

最初打算運用霍夫曼編碼，計算平均壓縮位元，確認是否與理想值接近，作為密碼強度的檢測，但在測試的過程中，發現平均壓縮位元的數值與理想值的差距偏小，不足以成為強度的確切檢測（不論長短，測試結果均微「強」，這很顯然不符合我們的預期）。

測試結果：

```
please type the word:302eeW0
the input string is :302eeW0
密碼熵值：2.52 位元
字符類型數：3
密碼強度：weak
total entropy:17.651484
```

```
please type the word:C7*(weqriiqkk  
the input string is :C7*(weqriiqkk  
密碼熵值：3.24 位元  
字符類型數：4  
密碼強度：strong  
total entropy:42.105716
```

資料庫處理：

功能概述：

由伺服器管理 JSON 格式的資料庫，根據客戶所輸入的指令，判斷其屬於要求何種功能，並執行相對應的函式。

技術細節：

操作指令庫的函示包含了五項，分別是 New 指令（新增欲增加的服務）、Get 指令（取得欲得知的服務）、Update 指令、Delete 指令以及 ls 指令。客戶與伺服器確立了連線後，由客戶端的終端機輸入指令，並顯示“Command executed successfully”（Command 與此處是通用，可自行換成 New, Get, Update 等詞彙）；伺服器端的終端機則會顯示收到了甚麼類型的指令，並給予對應的資訊，若收到 Get 指令，顯示對應的服務、以及其帳號、密碼，並顯示字符種類、密碼的強度、總密碼熵等資訊；Update 指令則是於客戶端顯示“Updated Successfully”訊息，終端機則是“Update Operation completed Successfully”。

測試結果：

1. New (以新增 “ILoveTrashFood” 此服務為例)

修改前：

```

{} test.json > {} getdunck > {} hungover
1  {
2    "appleTV": {
3      "haha": "imhandsome"
4    },
5    "IMAX": {
6      "leo": "bisdifj"
7    },
8    "LiveABC": {
9      "abcde": "oasdewrikgmverig"
10   },
11   "JackTV": {
12     "abcdef": "19gidoagh23iajgio;aet"
13   },
14   "George.com": {
15     "aodgsodjg": "19dfisaj29tua"
16   },
17   "getdunck": {
18     "hungover": "Gyv5P>yRMD4+m)n49ci15XuJ"
19   },
20   "shaco": {
21     "bluebox": "ajkfjlwetiojig94289ifhaioh"
22   }
23 }

```

修改後：

```

},
"shaco": {
  "bluebox": "ajkfjlwetiojig94289ifhaioh"
},
"ILOVETRASHFOOD": {
  "BECOMESOFAT": "CRUSHMYSELF"
}

```

1/2 之後（第一次報告完）的更新：

new 指令可以在同個服務裡面新增複數個帳號。例如說以下指令：

new getdunck duck duke

new getdunck notfound qwerty

new getdunck isjdia's aishdfweng

```

{
  "LiveABC": {
    "abcde": "aweighwet",
    "sidjfw": "ow8eyt2",
    "awieyt28": "we9gfuj8439ytus",
    "8waeyhf29": "8sdfh2803hf"
  },
  "getdunck": {
    "duck": "duke",
    "notfound": "qwerty",
    "isjdia's": "aishdfweng"
  },
  "Netflix": {
    "aisodjff": "sadowfjiwqeg",
    "asuidhf947u": "apsdugfu23g",
    "9w8yfagsu23": "8yerwh1"
  }
}

```

可以看到 getdunck 有多組帳號密碼

P.S. 1/2 之後的更新都會以這個檔案為範例

2. Get 指令 (以取得 “IMAX” 為例)

JSON 檔案內容：

```

() test.json > {} getdunck >  hungover
1  {
2    "appleTV": {
3      "haha": "imhandsome"
4    },
5    "IMAX": {
6      "leo": "bisdifj"
7    },
8    "LiveABC": {
9      "abcde": "oasdewrikgmverig"
10   },
11   "JackTV": {
12     "abcdef": "19gidoagh23iajgio;aet"
13   },
14   "George.com": {
15     "aodgsodjg": "19dfisaj29tua"
16   },
17   "getdunck": {
18     "hungover": "Gyv5P>yRMD4+m)n49ci15XuJ"
19   },
20   "shaco": {
21     "bluebox": "ajkfjllwetiojig94289ifhaioh"
22   }
23 }

```

客戶端：

```
Enter commend: get IMAX

IMAX:
  account: leo, password: bisdifj
密碼熵為： 2.52 位元
總密碼熵為： 17.65 位元
密碼強度建議： weak，建議修改

Enter commend: 
```

1/2 之後（第一次報告完）的更新：

我將 get 指令分為兩種，一種是只抓服務，不抓特定帳號密碼，會將所有服務裡的帳號密碼輸出，另一種是尋找特定帳號密碼並計算出密碼強度。

例如：get getdunck

```
Enter commend: get getdunck

getdunck:
  - account: duck, password: duke
  - account: notfound, password: qwerty
  - account: isjdia's, password: aishdfweng
```

他就會把所有在 getdunck 這個服務裡的帳號密碼輸出。

另外例如：get getdunck duck

```
Enter commend: get getdunck duck

getdunck:
  - account: duck, password: duke
密碼熵為： 2.00 位元
總密碼熵為： 8.00 位元
密碼強度建議： weak,建議修改
```

他就會尋找在 getdunck 服務裡的 duck 帳號。

3. Update 指令

服務更新前：


```
},  
"shaco": {  
  "bluebox": "ajkfj lwetiojig94289ifhaioh"  
},  
"ILOVETRASHFOOD": {  
  "BECOMESOFAT": "CRUSHMYSELF"  
}
```

服務更新後：

```
"shaco": {  
  "bluebox": "ajkfj lwetiojig94289ifhaioh"  
},  
"ILOVETRASHFOOD": {  
  "BECOMESOFAT": "STARTGOINGTOGYM"  
}
```

1/2 之後（第一次報告完）的更新：

新增 up 帳號密碼的身份認證，若想要更新一組帳號密碼，必須要先輸入原本該帳號的密碼才可以更新。

例如：up getdunck duck dukesohandsome

(此字串的第四個部分是想要更新的密碼)

```
Enter commend: up getdunck duck dukesohandsome  
  
Password:duke  
  
Please type in original password: duke  
  
- identity checked  
  
Updated successfully.
```

可以看到跳出的 Please type in original password:

只要輸入原本的密碼即可更新（我在這邊加入密碼提示，讓我在測試功能的時候不會需要查看檔案），若輸入錯的話則會回到 Enter command:

```
"getdunck": {  
  "duck": "dukesohandsome",  
  "notfound": "qwerty",  
  "isjdia's": "aishdfweng"  
},
```

此為更新完的檔案內容

4. Delete 指令

服務刪除前：

```
"shaco": {  
  "bluebox": "ajkfjlwetiojig94289ifhaioh"  
},  
"ILOVETRASHFOOD": {  
  "BECOMESOFAT": "STARTGOINGTOGYM"  
}
```

服務刪除後：

```
"shaco": {  
  "bluebox": "ajkfjlwetiojig94289ifhaioh"  
}
```

1/2 之後（第一次報告完）的更新：

跟 up 指令一樣，我加入了身份認證的功能。

例如：del getdunck duck

```
Enter commend: del getdunck duck
```

```
Password:dukesohandsome
```

```
Please type in original password: █
```

我想刪除 getdunck 裡面的 duck，就會需要輸入原本的 password 才可以刪除。

```
Please type in original password: dukesohandsome  
- identity checked  
-Deleted successfully.
```

```
"getdunck": {  
  "notfound": "qwerty",  
  "isjdia's": "aishdfweng"  
},
```

刪除完之後的檔案內容。

若 getdunck 裡面的帳號密碼都被刪完的話，del 也會把 getdunck 這個服務刪掉。

5. ls 指令(1/2 之後第一次報告完更新的):

```
Enter commend: ls
```

```
Services are:
```

- LiveABC
- getdunck
- Netflix

ls 指令會將所有的服務打印出來。

修改過程、試錯紀錄：

New 函式在一開始無法將帳號與密碼加入，經過推敲後發現是因為 cJSON_ReplaceItemObject 這個語法只能將已存在的物件做為目標，但是剛建立起的服務（在程式裡變數名為“service”）仍然為空物件，尚未初始化帳號和密碼。

甚至於一開始編譯時，終端機無法接受 cJON.h 標頭檔，詳細原因不清楚，但在改成 #include “cJSON.c” 後，便能夠成功執行。希望日後能夠翻閱相關的資料，釐清 vscode 的終端機是如何處理編譯器路徑以及標頭檔的位置，以免在第一步時就走的極為踉蹌，影響後續進度。

未來展望：

希望更加精細的控制服務、帳號、密碼，目前只能實現服務裡單一的帳號，若有多組帳號，目前還無法更加準確的選擇特定的帳號，不論是取得還是編輯，

現階段若使用 `get`，則會得到該服務裡所有的帳號。另外，也希望密碼的檢測更加精準，經過多次測試，發現結果僅有「弱」與「強」，「中等」並未顯現，但後來因為參數設定不當，將“entropy”改成“total_entropy”(代表該密碼所涵蓋的總資訊量)，密碼強度檢測更加精準。目前則是在思考該如何根據密碼給予提示，方便使用者在日後忘記密碼時能夠回想起來。

陳至恆後記：助教你好，我只想讓你知道，我對我跟這個學弟的合作感到失望。我們的分配是這樣的，他負責霍夫曼密碼處理，我負責伺服器 crud 的功能。我沒有想到這個的分配結果會那麼不均，程式幾乎都是我在寫。但真正讓我生氣的不是在分配，而是他的消極態度。我們有訂一天要把兩人的程式和在一起跑，當天要討論的時候才用 chatgpt 生給我，而且結束後才傳給我可用的，甚至也不告訴詳細函示要怎麼用。後來更新的事他也不聞不問，嘴上說說會幫忙寫新的功能，然後什麼話都不講直到要最後 demo 的時候。我叫他來做這個文件檔，做得也差強人意，後來我才有對這個文件檔做一些更新的東西和優化。他真的「程式」不足，敗事有餘。

總之，我們的分工比例時間大概是 7:1，我 7 他 1，請助教斟酌我們兩個分別的分數，否則難解我心頭之義憤填膺。感激不盡。